

Relatório Técnico: Sistemas Embarcados

Identificação do Aluno	Número USP	Data de Entrega
Giovanni Antonio Silva de Oliveira	14575064	21/05/2026

1. Identificação do Experimento

- **Título:** Atividade 2 - PWM
- **Disciplina:** PSI3441 - Arquitetura de Sistemas Embarcados
- **Professor:** Gustavo Pamplona Rehder

2. Respostas, Comentários e Análises

- Apenas para efeito de comentário, o código foi feito utilizando-se a biblioteca fornecida para PWM (que por si só, já é *hardCoded* na sua estrutura interna) e as orientações fornecidas para o uso do terminal serial (como TX, usando `printf`).
- A implementação feita pela interface do usuário baseia-se em interrupções (*callback*) causados pelo envio dos dados pelo terminal, que no caso a fonte provém do software *TeraTerm*.
- As interações com o programa baseiam-se:
 - Comando Ligar led
 - Comando Desligar led
 - Status das configurações atuais
 - Set de porcentagem de brilho

3. Código-Fonte

O código abaixo representa a implementação lógica realizada para o hardware embarcado.

```
#include <zephyr/kernel.h> // Funções básicas do Zephyr
(ex: k_msleep, k_thread, etc.)
```

```

#include <zephyr/device.h>           // API para obter e utilizar
dispositivos do sistema
#include <zephyr/drivers/gpio.h>     // API para controle de pinos de
entrada/saída (GPIO)
#include <zephyr/drivers/uart.h>     // Adicionado para a API de UART
por interrupção

#include <pwm_z402.h>                // Biblioteca personalizada com
funções de controle do TPM (Timer/PWM Module)~
#include <stdio.h>                   // Biblioteca Printf
#include <stdlib.h>                   // Comando atoi
#include <string.h>                   // tipo String

#define TPM_MODULE 1000              // Define a frequência do PWM fpwm =
(TPM_CLK / (TPM_MODULE * PS))
#define RX_BUF_SIZE 32              // Tamanho buffer recepção

#define DELTA_PRINT 500
#define DELTA_PWM 10

// #define DUTY_PASS 5

void printData();

void terminal_callback(const struct device *dev, void *user_data); //
função do Callback
void process_rx(char* msg); // Processa mensagem Callback

volatile uint16_t pwm_target_duty = 1; // volatile pois muda fora do
código
volatile bool pwm_enable = true;

int main(void)
{
    // Inicializa o módulo TPM2 com:
    // - base do TPMx
    // - fonte de clock PLL/FLL (TPM_CLK)
    // - valor do registrador MOD
    // - tipo de clock (TPM_CLK)
    // - prescaler de 1 a 128 (PS)
    // - modo de operação EDGE_PWM

```

```

    pwm_tpm_Init(TPM2, TPM_PLLFLL, TPM_MODULE, TPM_CLK, PS_128,
EDGE_PWM);

    // - modo TPM_PWM_L (nível baixo durante o pulso)
    pwm_tpm_Ch_Init(TPM2, 0, TPM_PWM_L, GPIOB, 18);
    pwm_tpm_Ch_Init(TPM2, 1, TPM_PWM_L, GPIOB, 19);

    const struct device *uart_dev =
DEVICE_DT_GET(DT_CHOSEN(zephyr_console)); // cria o header da UART
conectada ao console

    uart_irq_callback_user_data_set(uart_dev, terminal_callback,
NULL); // Para o dado UART Header, registra a função do Calback
    uart_irq_rx_enable(uart_dev); // Habilita apenas interrupção
para RX

    printk("=== LED PWM por terminal ===\n");
    printk("Digite:\n");
    printk("L ou l para ligar\n");
    printk("D ou d para desligar\n");
    printk("Pxxx para colocar salvar uma intensidade, em
porcentagem\n");
    printk("s ou S para status\n");
    printk("=====\n");

    //uint16_t duty = 0;

    //uint64_t delayPrint = k_uptime_get();
    uint64_t delayPWM = k_uptime_get();

    while (1)
    {
        uint64_t actualTime = k_uptime_get();

        if(actualTime - delayPWM >= DELTA_PWM)
        {
            if (pwm_enable)
            {
                pwm_tpm_CnV(TPM2, 0, pwm_target_duty);
                pwm_tpm_CnV(TPM2, 1, pwm_target_duty);
                //pwm_tpm_CnV(TPM2, 0, duty);
                //duty += DUTY_PASS;
                //if(duty > pwm_target_duty) duty = 0;
            }
        }
    }

```

```

    }
    else
    {
        pwm_tpm_CnV(TPM2, 0, 0);
        pwm_tpm_CnV(TPM2, 1, 0);
        //pwm_tpm_CnV(TPM2, 0, 0);
        //duty = 0;
    }
    delayPWM = k_uptime_get();
}

//if(actualTime - delayPrint >= DELTA_PRINT)
//{
//    printData();
//    delayPrint = k_uptime_get();
//}
}
return 0;
}

void terminal_callback(const struct device *dev, void *user_data) //
função do Callback
{
    static char rx_buffer[RX_BUF_SIZE];
    static size_t rx_idx = 0;
    uint8_t data;

    if(!uart_irq_update(dev)) return;

    while (uart_fifo_read(dev, &data, 1) == 1)
    {
        if(data == '\n' || data == '\r')
        {
            if (rx_idx > 0) // Checa final de texto e se
tem tamanho
            {
                rx_buffer[rx_idx] = '\0'; // Marca
final da String

                process_rx(rx_buffer); // Processa
                rx_idx = 0; // reinicia indexador
            }
        }
    }
}

```

```

        else if(rx_idx < (RX_BUF_SIZE - 1)) // checa se não vai
estourar o buffer
        {
            rx_buffer[rx_idx] = data;
            rx_idx++;
        }
    }
}

void process_rx(char* msg)
{
    if(msg[0] == '\0') return; // texto nulo

    if((msg[0] == 'L' || msg[0] == 'l') && msg[1] == '\0') // Liga
    {
        pwm_enable = true;
        printk("\nPWM Ativado!\n");
    }

    else if ((msg[0] == 'D' || msg[0] == 'd') && msg[1] == '\0') //
Desliga
    {
        pwm_enable = false;
        printk("\nPWM Desativado!\n");
    }

    else if ((msg[0] == 'S' || msg[0] == 's') && msg[1] == '\0') //
Desliga
    {
        printData();
    }

    else if (msg[0] == 'P' || msg[0] == 'p') // Porcentagem
    {
        uint8_t percent = (uint8_t)atoi(&msg[1]); // Converte o
valor do array, a partir do 2 endereço, em um numero inteiro

        if (percent >= 0 && percent <= 100)
        {
            pwm_target_duty =
TPM_MODULE*(uint16_t)percent/100;
            printk("\nDuty Cycle definido para: %d\n",
percent);

```

```

        }
        else printk("\nErro ao fixar a porcentagem. processado
%d\n", percent);
    }

    else printk("\ncomando %s desconhecido\n", msg);
}

void printData()
{
    static int counter = 0;

    printk("=== printk() - Status ===\n");
    printk("Contador: %d\n", counter);
    printk("Estado: %s\n", pwm_enable ? "LIGADO" : "DESLIGADO");
    printk("Intensidade: %d\n", pwm_target_duty);
    printk("=====\n");
    counter++;
}

```

4. Referências e Links Externos

- **Link do Repositório (GitHub/GitLab):**

<https://github.com/Djovanne/PSI3441---Arquitetura-de-Sistemas-Embarcados/tree/main/Atividade%202%20-%20PWM%20e%20Serial>